Lecture Notes: Hierarchical Navigable Small World Graphs

Approximate Nearest-Neighbor Search in Metric Spaces

Contents

1 Motivation and Big Picture **2**
1.1 The Spectrum of Approaches 2

2 The Skip-List Analogy **2**

3 From NSW to HNSW: Why Layers? **3**

4 Data Structure **3**
4.1 Layer Assignment 3
4.2 Key Parameters 4

5 The Five Algorithms **4**
5.1 FIND-NN: Greedy Descent Across Layers 4
5.2 BEAM-SEARCH: Single-Layer Beam Search 5
5.3 SEL-NEIGHBORS: Heuristic Neighbor Selection 6
5.4 INSERT: Building the Index 7
5.5 SEARCH: Answering K-NN Queries 8

6 Complexity Analysis **8**
6.1 Search: $O(\log N)$ 8
6.2 Construction: $O(N \log N)$ 8
6.3 Memory 9

7 Putting It All Together: A Worked Example **9**

8 Parameter Tuning in Practice **9**

9 Summary **10**

1 Motivation and Big Picture

Nearest-neighbor search is a core primitive: retrieval-augmented generation, recommendation systems, image similarity, vector databases—all reduce to "given a query t, find the K database points closest to t under some distance d." Exact search scans all N points per query. We want a data structure that answers K-NN queries much faster, with high probability of returning the true nearest neighbors.

HNSW (Malkov & Yashunin, 2016) achieves this by building a multi-layer proximity graph whose structure mirrors a skip list. These notes develop the algorithm from that analogy.

2 The Skip-List Analogy

A **skip list** organizes N sorted elements into multiple linked lists layered on top of each other. Every element lives on layer 0 (the base linked list). Each element is independently promoted to layer 1 with probability 1/2, to layer 2 with probability 1/4, and so on. To search for value x:

1. Start at the leftmost element of the top layer.
2. Move right as long as the next element is $\leq x$.
3. Drop down one layer and repeat.

The upper layers are *express lanes*—sparse, long-range connections that skip over most of the base list, delivering logarithmic search cost.

Key Insight 1. HNSW is the generalization of the skip list to an arbitrary metric space. Sorted linked lists become *proximity graphs*; "move to the next larger element" becomes "move to the nearest neighbor."

Skip list	HNSW
Sorted linked list at layer ℓ	Proximity graph at layer ℓ
Left/right neighbors	M approximate nearest neighbors
Promotion probability 1/2	Promotion probability $1/M$
"Move right"	"Move to closer neighbor" (greedy)
Drop to next layer	Drop to next layer

In one dimension with $M = 2$, HNSW recovers the exact skip list: the heuristic neighbor selection (Section 5.3) picks the left and right neighbors as the sole connections on layer 0.

3 Why Layers?

Consider a flat proximity graph where every element connects to its M nearest neighbors. A greedy hill-climbing search on such a graph works, but the cost grows poorly with N because a few "hub" nodes accumulate many connections (other elements nominate them as nearest neighbors), and every search path is forced through these hubs.

HNSW's fix: **separate long-range links from short-range links into different layers, each with degree bounded by M.** Long-range links live on upper layers (sparse, few elements). Short-range links live on lower layers (dense, all elements). Bounded degree at every layer ensures search work does not blow up through hub nodes.

> Think of it like a road network. Layer 0 is city streets (all intersections, short blocks). Layer 1 is arterial roads. Layer 2 is highways. A good route planner takes the highway to the right city, the arterial to the right neighborhood, then local streets to the address. Starting on city streets from a random city would be absurdly slow.

4 Data Structure

An HNSW index stores:

- A set of elements $\{x_1, \ldots, x_N\}$ in a metric space with distance d.
- For each element x and each layer ℓ it participates in: a neighbor list $\text{neigh}[x, \ell]$ of size $\leq M$ (or $\leq 2M$ on layer 0).
- A global **entry point** *ep*: the element residing on the highest layer.
- The current maximum layer L.

4.1 Layer Assignment

Each inserted element w is assigned a maximum layer lw by independent Bernoulli coin flips with success probability $1/M$:

> Set $lw \leftarrow 0$. Flip a biased coin (heads with probability $1/M$). While it comes up heads, increment lw and flip again.

This gives $lw \sim \text{Geom}(1 - 1/M) - 1$, so:

$$\Pr[lw \geq k] = (1/M)^k.$$

Element w then lives on layers $0, 1, \ldots, lw$.

Example 1. With $M = 16$: the probability of reaching layer 1 is $1/16 \approx 6\%$, layer 2 is $1/256 \approx 0.4\%$, layer 3 is 0.002%. With $N = 10^6$ elements, the expected number on layer 2 is about 4000, on layer 3 about 250. The hierarchy is very shallow—typically 4–6 layers for a million-element index.

Why this formulation? The coin-flip description directly mirrors skip-list promotion and makes it immediately clear that layers are geometrically thinned. The paper uses the equivalent formula $lw = \lfloor -\ln(\text{Unif}(0,1)) \cdot m_L \rfloor$ with $m_L = 1/\ln M$, which produces the same geometric distribution.

4.2 Key Parameters

Parameter	Typical range	Role
M	5–48	Max neighbors per layer; controls memory and graph quality
$M_{\max 0} = 2M$	10–96	Max neighbors on layer 0 (denser base layer)
$ef_{\text{Construction}}$	100–500	Beam width during index build; controls recall of built graph
ef	$\geq K$	Beam width at query time; the runtime recall/speed knob

Memory per element: roughly $(M_{\max 0} + m_L \cdot M) \times 4$ bytes for neighbor indices, typically 60–450 bytes.

5 The Five Algorithms

5.1 find-nn: Greedy Descent Across Layers

Purpose. Given a query t and a target layer ℓ, find the node on layer ℓ that is (approximately) closest to t. This is the *cheap* search used on upper layers, committing to a single current node at each step with no candidate queue.

Algorithm. Start at the global entry point ep on the top layer L. At the current layer, scan the current node's neighbors; if any neighbor is closer to t than the current node, move there and repeat. When no improvement is possible (local minimum on this layer), descend to the next layer. Repeat until layer ℓ is reached.

Algorithm 1 FIND-NN — greedy descent across all layers

```
function FIND-NN(t, ℓ)
    nn ← ep
    ℓ′ ← L
    while ℓ′ ≥ ℓ do
        repeat
            done ← ⊥
            for v ∈ N(nn, ℓ′) do
                if d(v, t) < d(nn, t) then
                    nn ← v
                    done ← ⊤
                end if
            end for
        until done
        ℓ′ ← ℓ′ − 1
    end while
    return nn
end function
```

Invariant. The local minimum found on layer $\ell + 1$ serves as the starting point for layer ℓ. Because upper layers are sparser, their local minima are geometrically closer to t than a random start would be—each descent narrows the search region.

> This is exactly skip-list descent: on an upper layer you take big leaps (long-range edges), narrowing the search region. On lower layers the edges are shorter, so you land closer to the target. There is no candidate queue—you commit to one current node at a time. This is fast but can get trapped in local minima on dense layers, which is why layer 0 uses BEAM-SEARCH instead.

5.2 beam-search: Single-Layer Beam Search

Purpose. Search a *single* layer for the *ef* closest elements to query t, starting from entry point *ep*. This is the high-recall operation used on layer 0 during queries and on every layer during index construction.

Data structures. Three collections are maintained simultaneously:

Q **(candidates)** A min-heap ordered by $d(\cdot, t)$: the search frontier of nodes whose neighborhoods have not yet been explored.

W **(result set)** A max-heap ordered by $d(\cdot, t)$, capped at size *ef*: the running set of the *ef* closest nodes seen so far. The max-heap gives fast access to the *worst* current result.

D **(visited)** A hash set. Every node added to Q is immediately added to D and will never be added again.

Loop. While the nearest candidate in Q is closer to t than the worst element in W:

1. Pop the nearest candidate q^* from Q.
2. Add q^* to W; if $|W| > ef$, evict the farthest element from W.
3. For each neighbor v of q^* on this layer: if $v \notin D$ and v is no farther than the current worst in W (or W is not yet full), add v to both Q and D.

Termination condition: if the nearest candidate is already farther from t than our worst known result, nothing in Q (nor anything reachable from it) can improve W, so we stop.

> Think of W as your shortlist and Q as your to-do list. You keep exploring the most promising lead (minimum of Q), updating the shortlist when you find something better than its current worst entry. You stop when even the most promising lead can't beat your worst shortlist entry.

Why the visited set D? In a graph, the same node v can appear as a neighbor of multiple different candidates in Q. Without D, you would recompute $d(v, t)$ and re-examine v's neighbors redundantly through every such path. D ensures each node is processed at most once.

Algorithm 2 BEAM-SEARCH — single-layer beam search

```
function BEAM-SEARCH(t, ep, ℓ, ef)
    D ← {ep}                                                  ▷ visited
    Q ← {ep}                                                  ▷ candidates
    W ← {ep}                                                  ▷ result set
    while min_{q∈Q} d(q,t) < max_{w∈W} d(w,t) do
        q* ← arg min_{q∈Q} d(q,t)
        Q ← Q \ {q*}
        W ← W ∪ {q*}
        if |W| > ef then
            remove arg max_{w∈W} d(w,t) from W
        end if
        for v ∈ N(q*, ℓ) do
            if v ∉ D and d(v,t) ≤ max_{w∈W} d(w,t) then
                Q ← Q ∪ {v}
                D ← D ∪ {v}
            end if
        end for
    end while
    return W
end function
```

The ef parameter. ef is the recall/speed knob. Small ef (e.g., $ef = K$): fast, lower recall. Large ef (e.g., $ef = 500$): slower, near-perfect recall. During index construction $ef_{\text{Construction}}$ plays this role; at query time the user sets ef freely.

5.3 sel-neighbors: Heuristic Neighbor Selection

Purpose. Given a set of candidate neighbors W for element t, select the best $n \leq M$ of them. The naive approach—pick the n closest—has a fatal flaw.

The cluster-boundary problem. Suppose t sits at the boundary between two dense clusters A and B. The M closest points to t might all be in cluster A, leaving t with no connections to cluster B. Any search that reaches t from the B side gets stuck.

The heuristic. Process candidates nearest-first. Add candidate w^* to the result set N if and only if w^* is *closer to t than to any element already in N*:

$$d(w^*, t) < \min_{v \in N} d(w^*, v).$$

Otherwise, send w^* to the discard pile D. After exhausting W, if $|N| < n$, fill remaining slots from D nearest-first.

Geometric intuition. The condition asks: "Is w^* closer to t than to any of my already-chosen neighbors?" This rejects candidates that are "overshadowed" by an existing neighbor

Algorithm 3 SEL-NEIGHBORS — heuristic neighbor selection

```
function SEL-NEIGHBORS(t, W, ℓ, n)
    N ← ∅
    D ← ∅                                          ▷ discarded
    while W ≠ ∅ do
        w* ← arg min_{w∈W} d(w, t)
        W ← W \ {w*}
        if d(w*, t) < min_{v∈N} d(w*, v) then
            N ← N ∪ {w*}
        else
            D ← D ∪ {w*}
        end if
        if |N| ≥ n then
            return N
        end if
    end while
    while |N| < n and D ≠ ∅ do
        w* ← arg min_{w∈D} d(w, t)
        D ← D \ {w*}
        N ← N ∪ {w*}
    end while
    return N
end function
```

in the relative neighborhood graph sense. The result is angularly spread connections—roughly one per direction—approximating the Delaunay graph.

Example 2. In $\mathbb{R}^2$ with $M = 4$: if the four nearest candidates to t are all in direction $(1, 0)$, only the closest one is added. The next candidate in direction $(-1, 0)$ passes the test (it is closer to t than to the first chosen neighbor) and is added. The result has diverse angular coverage.

Example 3 (1D reduction)**.** In one dimension with $M = 2$: the heuristic selects exactly the left neighbor and the right neighbor of t, recovering the linked-list structure of a skip list. HNSW on the real line *is* a skip list.

Fallback. If strict diversity leaves N underfull (e.g., t is near a boundary of the dataset), discarded candidates fill the remaining slots. This keeps the graph well-connected even in sparse regions.

5.4 insert: Building the Index

Overview. Inserting element w involves: (1) assign a layer lw by coin flips, (2) for each layer from lw down to 0, find good candidates and wire bidirectional edges.

Algorithm 4 INSERT — insert element into HNSW

```
function INSERT(w, M, ef)
    lw ← 0
    P ←$ Bern(1/M)
    while P = 1 do
        lw ← lw + 1
        P ←$ Bern(1/M)
    end while
    while lw ≥ 0 do
        n* ← FIND-NN(w, lw)
        W ← BEAM-SEARCH(w, n*, lw, ef)
        N ← SEL-NEIGHBORS(w, W, lw, M)
        for v ∈ N do
            neigh[w, lw] ← neigh[w, lw] ∪ {v}
            neigh[v, lw] ← neigh[v, lw] ∪ {w}
        end for
        for v ∈ N ∪ {w} do
            if |neigh[v, lw]| > M then
                neigh[v, lw] ← SEL-NEIGHBORS(w, neigh[v, lw], lw, M)
            end if
        end for
        lw ← lw − 1
    end while
end function
```

Step 1: Layer assignment. Flip Bern$(1/M)$ coins and count heads before the first tail—that count is lw. This is exactly skip-list promotion. With $M = 16$ and $N = 10^6$ elements, the expected max layer is $\log_{16}(10^6) \approx 5$.

Step 2: Greedy descent to layer lw. Call FIND-NN to descend from the top layer L to layer lw, obtaining an approximate entry point n^*. If $lw \geq L$, skip this step—w becomes the new entry point.

Step 3: Layer-by-layer connection. For $\ell = lw$ down to 0:

1. BEAM-SEARCH$(w, n^*, \ell, ef_{\text{Construction}})$: find the $ef_{\text{Construction}}$ nearest nodes to w on layer ℓ.
2. SEL-NEIGHBORS(w, W, ℓ, M): select the best M diverse neighbors N.
3. Add bidirectional edges: $\text{neigh}[w, \ell] \leftarrow N$, and for each $v \in N$: $\text{neigh}[v, \ell] \leftarrow \text{neigh}[v, \ell] \cup \{w\}$.
4. **Shrink overfull neighbors:** for each $v \in N$, if $|\text{neigh}[v, \ell]| > M$ (or $> 2M$ on layer 0), run SEL-NEIGHBORS to trim v's list back to the limit.
5. Update n^* to the best candidate from W as the entry point for the next lower layer.

Step 4: Update global state. If $lw > L$, set $L \leftarrow lw$ and $ep \leftarrow w$.

Each insertion is essentially a search followed by bounded heuristic work. The key asymmetry: inserting w may push an existing element v over its degree limit. We trim v's neighbors with SEL-NEIGHBORS rather than simple truncation, preserving graph quality. This makes HNSW a *dynamic* index—insertions maintain structural quality incrementally.

Edge case: first element. If the index is empty, w simply becomes ep with $L = 0$ and no connections.

5.5 search: Answering K-NN Queries

Overview. Given query t and parameters $ef \geq K$:

1. **Greedy descent** (layers L down to 1): call FIND-NN to cheaply navigate to the right region, using pure greedy hill-climbing with no beam.
2. **Beam search on layer 0**: call BEAM-SEARCH$(t, n^*, 0, ef)$ to find the ef nearest candidates among all N elements.
3. **Select top K**: return the K nearest from W.

Algorithm 5 SEARCH — K-nearest neighbor query

```
function SEARCH(t, ef, K)
    ℓ ← L                                  ▷ start at the top layer
    n* ← FIND-NN(t, 0)                     ▷ greedy descent to layer 0
    W ← BEAM-SEARCH(t, n*, 0, ef)
    return K nearest elements of W
end function
```

Why only beam search on layer 0? Layer 0 is the only layer containing *all* N elements. Upper layers are sparse subsets; the true nearest neighbor may not appear on any upper layer. The greedy descent on upper layers is a cheap way to find a good starting point, after which beam search does the precise work.

The *ef* knob. Setting $ef = K$ is the fastest (minimum exploration) but may miss some true nearest neighbors. Setting $ef = 200$ with $K = 10$ examines 200 candidates to confidently return the top 10. The user trades query latency for recall.

Example 4. With $N = 10^6$ vectors, $M = 16$, $K = 10$, and $ef = 100$: empirically (Figure 11 of the paper), the ef needed for 95% recall is roughly constant as N grows from 10^4 to 10^6, validating the logarithmic search cost.

6 Complexity Analysis

6.1 Search

Under the idealized assumption that each layer is an exact Delaunay graph with bounded average degree (which SEL-NEIGHBORS approximates):

- There are $\log_M N$ layers on average (from the geometric layer distribution).
- On each layer, the greedy path terminates after a bounded expected number of steps, because bounded degree and geometric convergence of distances prevent long chains of improvements.
- The number of elements per layer forms a geometric series: $N, N/M, N/M^2, \ldots$, so the total work across layers grows as $\sum_{k=0}^{\infty}(1/M)^k = M/(M-1)$—a constant independent of N.

Total expected distance computations for FIND-NN: logarithmic in N. Beam search on layer 0 adds $ef \cdot M$ additional comparisons, which is a user-controlled constant independent of N.

6.2 Construction

Each of the N insertions performs one logarithmic-cost search (via FIND-NN per layer) plus one BEAM-SEARCH and one SEL-NEIGHBORS per layer, each bounded by $ef_{\text{Construction}} \cdot M$ work. This gives logarithmic cost per insertion, hence $N \log N$ total.

6.3 Memory

Each element on layer $\ell > 0$ stores up to M neighbor indices; layer 0 stores up to $2M$. Expected total neighbors per element:

$$2M + \sum_{k=1}^{\infty} M \cdot (1/M)^k \approx 2M + \frac{M}{M-1} \approx 2M.$$

So roughly $2M \times 4 = 8M$ bytes per element for neighbor storage (assuming 4-byte integer IDs), plus the raw vectors.

7 Putting It All Together: A Worked Example

Suppose we build an index with $M = 4$ and $ef_{\text{Construction}} = 20$, inserting points one by one.

Insertion 1: Element x_1 is inserted into an empty index. x_1 becomes the entry point ep with $L = 0$. Coin flip: tails, so $lw = 0$.

Insertion 2: Element x_2. Coin flip: tails, so $lw = 0$. BEAM-SEARCH on layer 0 from $ep = x_1$ finds $W = \{x_1\}$. SEL-NEIGHBORS returns $N = \{x_1\}$. Add bidirectional edge $x_2 \leftrightarrow x_1$.

Insertion 5 (promoted to layer 1): Element x_5. First flip: heads ($lw \leftarrow 1$). Second flip: tails. So $lw = 1$. Since $1 > L = 0$: set $L = 1$, $ep = x_5$. On layer 1, x_5 has no other elements—no edges added. On layer 0, normal insertion with bidirectional edges to the nearest found neighbors.

Later query: SEARCH$(t, ef = 10, K = 3)$. FIND-NN starts at x_5 on layer 1, descends to layer 0. BEAM-SEARCH with $ef = 10$ returns 10 candidates. Return the 3 nearest.

8 Parameter Tuning in Practice

M Default 16 for most cases. Use smaller values (5–12) for low-dimensional data or when memory is tight. Use larger values (32–64) for high-dimensional data or when maximizing recall.

$ef_{\text{Construction}}$ Default 200. Doubling this roughly halves graph-quality errors but doubles build time. Set this once at build time and cannot be changed afterward without rebuilding.

ef **(query time)** Start at $ef = K$, increase until the recall target is met. The recall–latency curve is typically steep at first then flattens; there is usually a sweet spot.

Remark 1. $ef_{\text{Construction}}$ is baked into the graph structure at build time. ef can be changed freely per query without rebuilding the index. If recall is insufficient, increase ef first before deciding to rebuild.

9 Summary

Property	Value
Analogy	Skip list over a metric space
Core idea	Separate long- and short-range links by layer
Search cost	Logarithmic in N
Build cost	$N \log N$
Memory	$\sim 8M$ bytes/element for neighbor indices
Key algorithmic insight	SEL-NEIGHBORS gives directional diversity
Runtime knob	ef (query-time beam width)
Build knob	$ef_{\text{Construction}}$, M

HNSW achieves its efficiency through three reinforcing design choices:

1. **Layered hierarchy** (skip-list structure) enables cheap coarse navigation before the expensive fine search.
2. **Heuristic neighbor selection** (SEL-NEIGHBORS) ensures graph connectivity across cluster boundaries, preventing search from getting trapped.
3. **Bounded degree** at every layer prevents search cost from growing through high-degree hub nodes.

Reference: Yu. A. Malkov, D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs," arXiv:1603.09320v4 (2018).